

Detection of AI-Generated Arabic Text using Pyspark

Moutzz Abokhashabh

Introduction

project objectives:

- ▶ Design and implement a distributed data processing pipeline for Arabic text analysis using Apache Spark
- ▶ To develop comprehensive Arabic text preprocessing functions, including diacritic removal, character normalization, stopwords filtering, and stemming, implemented as distributed Spark UDFs for parallel execution
- ▶ To engineer hybrid features that combine TF-IDF vectors,
- ▶ To train and evaluate multiple classification models (Logistic Regression, Random Forest, Linear SVM, and Naive Bayes) on a balanced dataset of human and AI-generated Arabic abstracts
- ▶ To implement a streaming module using Spark Structured Streaming that demonstrates real-time inference capability on simulated file-based input streams

Dataset Description

Source: The dataset is KFUPM-JRCAL/arabic-generated-abstracts loaded from Hugging Face.

Structure: The dataset contains three splits:

- ▶ by_polishing: 2,851 rows
- ▶ from_title: 2,963 rows
- ▶ from_title_and_content: 2,574 rows

Dataset Description

Label	Generated By	Count	Percentage
0 (AI)	allam	8,388	20.0%
0 (AI)	jais	8,388	20.0%
0 (AI)	llama	8,388	20.0%
0 (AI)	openai	8,388	20.0%
1 (Human)	human	8,388	20.0%

Feature Engineering

Feature	Code	Description	Computation
F023	avg_syllables_per_word	Mean syllable count per word	Cluster-based vowel detection (Arabic long/short vowels)
F046	num_nouns	Noun frequency per abstract	camel-tools POS tagging (fallback: heuristic with ج prefix + common suffixes)

Preprocessing Functions

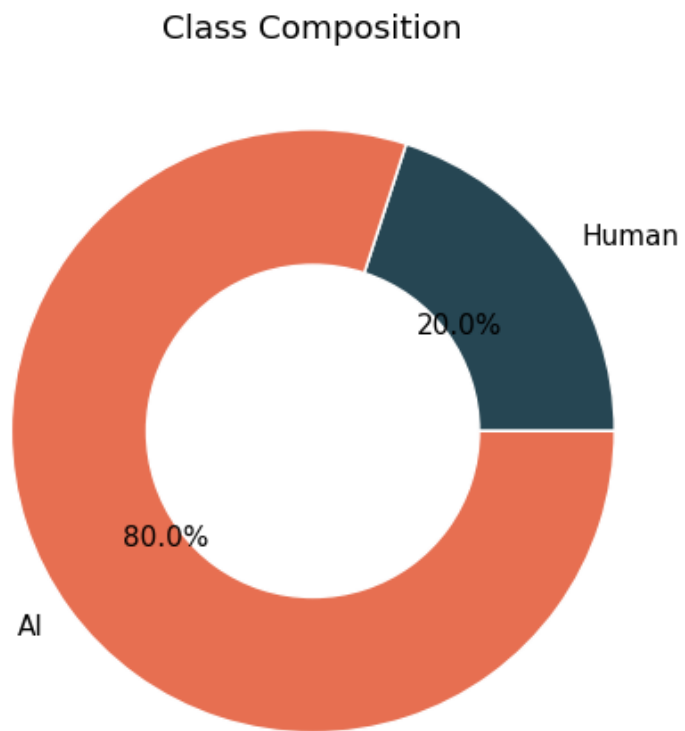
Step 1: Remove diacritical marks (tashkeel)

Step 2: Normalize character variants

Step 3: Filter Arabic stopwords by using NLTK

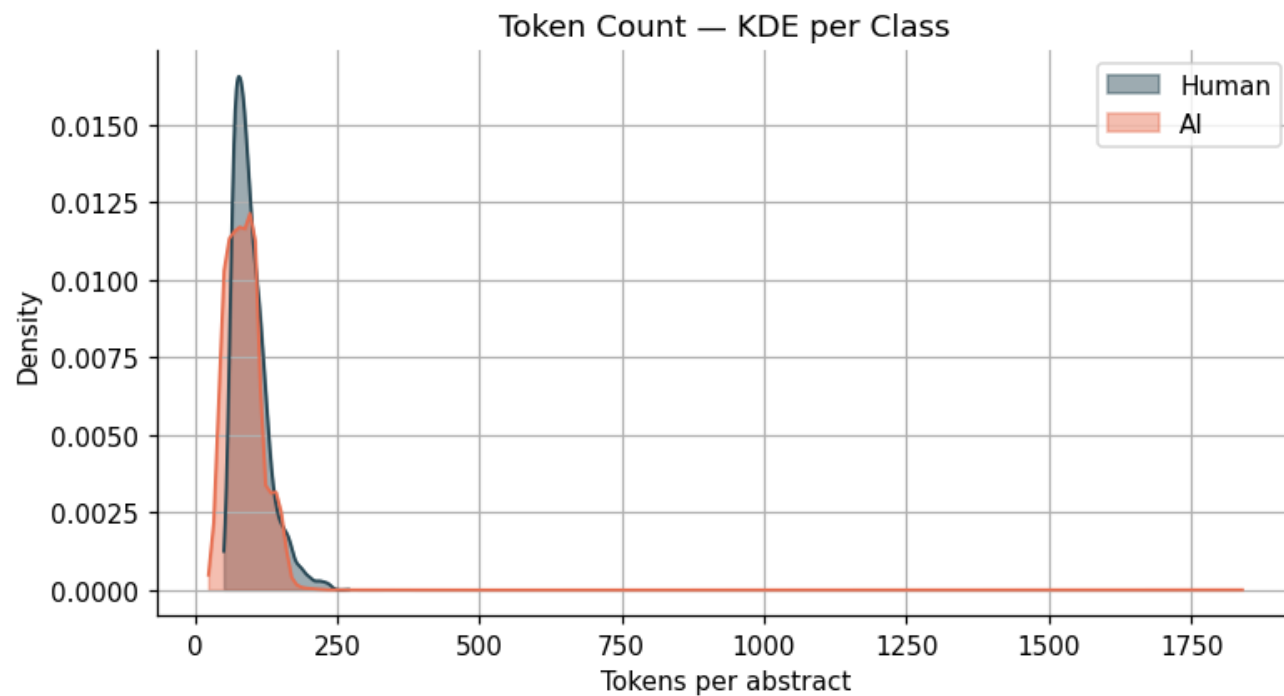
Exploratory Data Analysis

Class Distribution



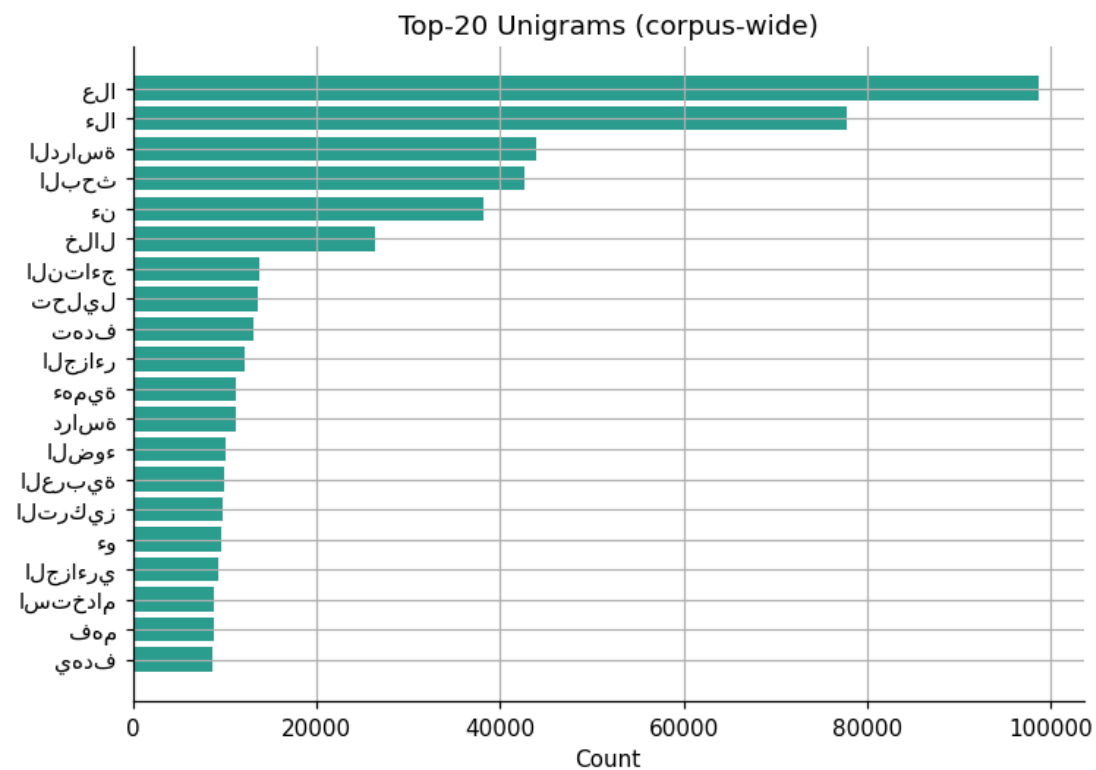
Exploratory Data Analysis

Token Density



Exploratory Data Analysis

Top Unigrams



Model Training Configuration

Split	Percentage	Row Count
Training	70%	29,345
Validation	15%	6,430
Test	15%	6,161

Results

Model	Accuracy	F1-Score	AUC-ROC
Logistic Regression	0.9802	0.9804	0.9955
Linear SVM	0.9750	0.9751	0.9946
Naive Bayes	0.9087	0.9137	0.7275
Random Forest	0.8123	0.7396	0.9872

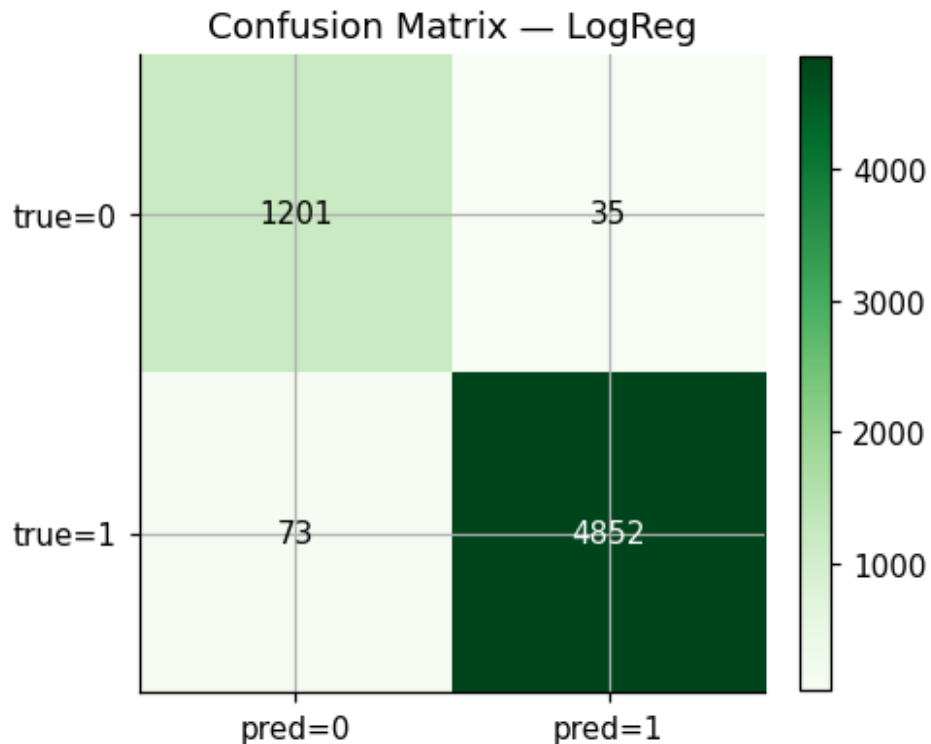
Evaluation

Best Model (Logistic Regression) Performance

Metric	Score
Accuracy	98.25%
F1-Score	0.9826
AUC-ROC	0.9961

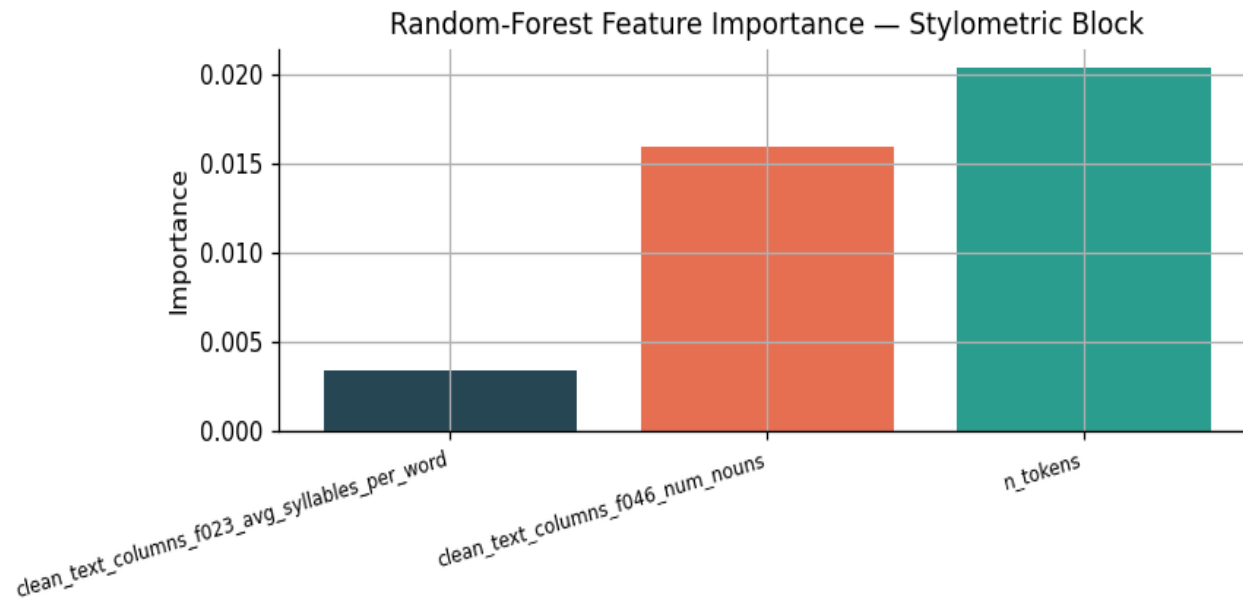
Confusion Matrix

The model shows excellent performance with only 35 false positives (AI misclassified as human) and 73 false negatives (human misclassified as AI). The low error rates indicate strong discriminative capability.



Feature Importance

Token count (`n_tokens`) showed the highest importance among stylometric features, followed by noun count. Average syllables per word contributed minimally, suggesting that syllable-level features may be less discriminative than lexical density measures for this dataset.



Streaming Implementation

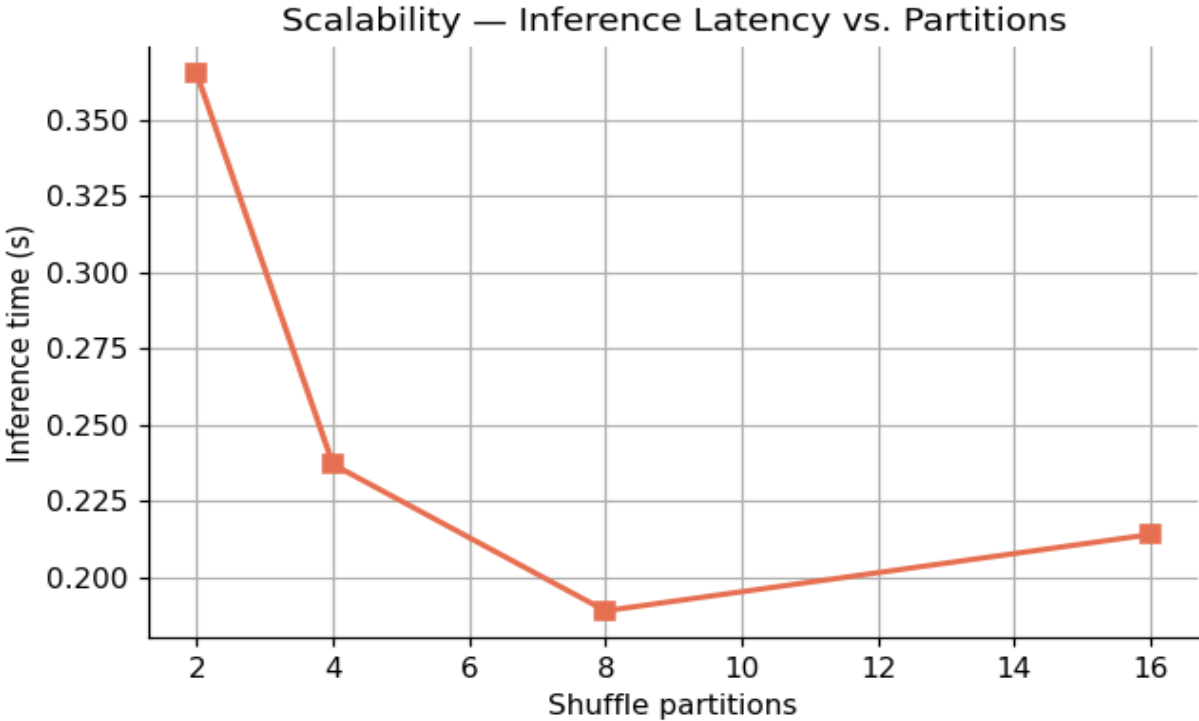
- ▶ JSON Files
- ▶ Spark Structured Streaming
- ▶ Preprocessing UDFs ↓
- ▶ TF-IDF Pipeline Transform
- ▶ Model Inference (Logistic Regression)
- ▶ Parquet Output

Streaming Implementation

Metric	Value
Total records	50
Batches	5
Processing mode	Append, 3-second trigger
Output format	JSON (partitioned)
Accuracy (simulated)	95.0%

Scalability Benchmark

Partitions	Time (seconds)	Efficiency
2	0.365	Baseline
4	0.237	Optimal
8	0.189	Improved
16	0.214	Overhead increase



Summary

This project successfully implemented a distributed big data pipeline for Arabic AI-text detection using Apache Spark. The Logistic Regression model, trained on a hybrid feature set combining TF-IDF and stylometric features (average syllables per word and noun count), achieved excellent performance (98.25% accuracy, 0.9826 F1-score, 0.9961 AUC). The streaming implementation demonstrated real-time inference capability, and scalability analysis identified optimal parallelism at 8 partitions.